

Backups using rsync and comparison with duplicity

Practical Training Documentation

Local Backup · Incremental Snapshots · Disaster Recovery · Tool Comparison

Environment	Details
Platform	Oracle VirtualBox
Operating System	Parrot OS 7.1 Security Edition (KDE)
Primary Tool	<code>rsync</code>
Secondary Tool	<code>duplicity</code>
Backup Scope	Local — within Parrot OS VM

Table of Contents

- Step 1 — Content of Source Files
- Step 2 — Install rsync
- Step 3 — Backup Script: Creation, Logic and Permissions
- Step 4 — Run First Backup Manually
- Step 5 — Schedule Backup Using Crontab
- Step 6 — Disaster Recovery
- Step 7 — Are There Other Options? rsync vs duplicity

Introduction

This document covers practical local backup implementation using two complementary tools: rsync and duplicity. All operations are performed inside a Parrot OS virtual machine, simulating a realistic environment where sensitive data requires regular, automated backup.

The scenario involves a directory named S-PII.info — representing a collection of Personally Identifiable Information (PII) files that require protection. The backup structure is as follows:

Path	Role
/home/user/S-PII.info (d)	Source directory containing the sensitive files to protect
/backups/current_backup (d)	Always holds the latest version of all files
/backups/backup_history (d)	Stores complete timestamped snapshots of every backup run
/var/log/backup.log (f)	Contains information about out backups

The three source files being backed up are:

- /home/user/S-PII.info/clients.txt
- /home/user/S-PII.info/phone_numbers.txt

- /home/user/S-PII.info/cl_id.txt

In a real-world environment, PII data is subject to strict regulatory requirements (GDPR, HIPAA, PCI-DSS). We should take it into account and backup such data in encrypted, access-controlled, and auditable way.

Step 1

Content of Source Files

Step 1 — Content of Source Files

Before running any backup, it is important to verify the content of the source files. This establishes a baseline — a known state we can compare against after backups and restores to confirm data integrity.

clients.txt — initial content

```
[user@parrot]~  
$ cat /home/user/S-PII.info/clients.txt  
John  
Jane  
Robert  
Ivan
```

phone_numbers.txt — initial content

```
[user@parrot]~  
$ cat /home/user/S-PII.info/phone_numbers.txt  
+1-555-0101  
+1-555-0102  
+1-555-0103  
+1-555-0104
```

cl_id.txt — initial content

```
[user@parrot]~  
$ cat /home/user/S-PII.info/cl_id.txt  
CLT-001  
CLT-002  
CLT-003  
CLT-004
```

TIP for me and other learners: Always document your baseline. If you cannot describe the state of data before a backup, you cannot verify the backup was successful.

Step 2

Install rsync

Step 2 — Install rsync

rsync (Remote Sync) is a fast and versatile file copying tool. It is the industry standard for local and remote file synchronisation, and is used extensively in backup automation, server mirroring, and penetration testing for file transfer.

rsync Flag	Meaning
-a	Archive mode — preserves permissions, timestamps, symlinks and ownership
-v	Verbose — prints each file being transferred
--delete	Removes files from destination that no longer exist in source
--backup	Moves overwritten files to a separate directory before replacing them

Install rsync using the package manager:

Command — install rsync

```
[user@parrot]~$ sudo apt install rsync
```

Verify the installation was successful:

Output — rsync version confirmed

```
[!] Scanning application launchers
Removing duplicate or broken launchers...
[!] Launchers have been successfully updated!
```

Step 3

Backup Script: Creation, Logic and Permissions

Step 3 — Backup Script: Creation, Logic and Permissions

Since backups should run automatically and consistently, they are managed through a shell script executed by a cron job. The script is stored in `/home/user` — not the standard Linux location for custom system administration scripts (which is `/usr/local/sbin/`), but I've placed it there for my better performance.

3.1 — Creating the Script

Command — create the script file

```
[X]-[user@parrot]-[/etc]$ sudo vim /home/user/rsync_scr.cl
```

Script contents in VIM editor

```
File Edit View Bookmarks Plugins Settings Help
1 #!/bin/bash
2
3 # — Configuration —
4 SOURCE="/home/user/S-PII.info/"
5 CURRENT="/backups/current_backup"
6 HISTORY="/backups/backup_history"
7 TIMESTAMP=$(date +%Y-%m-%d_%H-%M-%S)
8 LOGFILE="/var/log/backup.log"
9
10 cp -r "$CURRENT/." "$HISTORY/$TIMESTAMP"
11
12 rsync -av --delete "$SOURCE" "$CURRENT" >> $LOGFILE 2>&1
13
14 echo "[$TIMESTAMP] Backup completed — snapshot saved to $HISTORY/$TIMESTAMP" >> $LOGFILE
15 echo "—————" >> $LOGFILE
```

3.2 — How the Script Works

The script follows a three-phase approach on every execution:

Phase	Script Line	What It Does
1 — Snapshot	<code>cp -r "\$CURRENT/" "\$HISTORY/\$TIMESTAMP"</code>	Copies ALL current files into a new timestamped folder before anything is overwritten
2 — Sync	<code>rsync -av --delete "\$SOURCE" "\$CURRENT"</code>	Updates current backup to exactly match source — including deletions
3 — Log	<code>echo ... >> \$LOGFILE</code>	Appends a timestamped record of the backup event to the log file

The result of this approach is a clean and predictable backup structure:

- `current_backup` — always contains only the latest version of all files
- First directory in `backup_history`— complete snapshot of ALL files at that exact moment
- Second directory in `backup_history` — complete snapshot some time (5 minutes, according to the script) later

The `TIMESTAMP` variable uses `$(date +"%Y-%m-%d_%H-%M-%S")` which generates a unique folder name for every run — making every snapshot individually identifiable and sortable by date.

3.3 — Granting Execute Permission

For the script to be executed by the cron scheduler, it must have execute permission. The `chmod +x` command grants this:

Command — `chmod +x` to grant execute permission

```
[x]~[user@parrot]~[/etc]
→ $sudo chmod +x /home/user/rsync_scr_cl
```

The `x` permission flag is what tells Linux this file is an executable script, not just a text file. Without it, cron will fail silently.

Step 4

Run First Backup Manually

Step 4 — Run First Backup Manually

Before scheduling any automated task, it is essential to verify that the script executes correctly. Running it manually allows us to observe the output directly and catch any errors before automation begins.

Command — run the backup script manually

```
[user@parrot]~[/etc]
→ $sudo /home/user/rsync_scr_cl
```

Output — first backup results showing files synced and directories created

```
[user@parrot]~[/etc]
└─$ ls -la /backups/current_backup/
total 12
drwxrwxr-x 1 user user 74 May 13 17:52 .
drwxr-xr-x 1 user user 56 May 13 17:46 ..
-rw-rw-r-- 1 user user 32 May 13 17:50 cl_id.txt
-rw-rw-r-- 1 user user 24 May 13 17:48 clients.txt
-rw-rw-r-- 1 user user 48 May 13 17:52 phone_numbers.txt
[user@parrot]~[/etc]
└─$ ls -la /backups/backup_history
total 0
drwxr-xr-x 1 user user 76 May 13 18:15 .
drwxr-xr-x 1 user user 56 May 13 17:46 ..
drwxr-xr-x 1 root root 0 May 13 18:15 2026-05-13_18-15-38
[user@parrot]~[/etc]
└─$ cat /var/log/backup.log
sending incremental file list
./
cl_id.txt
clients.txt
phone_numbers.txt

sent 378 bytes  received 76 bytes  908.00 bytes/sec
total size is 104  speedup is 0.23
[2026-05-13_18-15-38] Backup completed - snapshot saved to /backups/backup_history/2026-05-13_18-15-38
```

Step 5

Schedule Backup Using Crontab

Step 5 — Schedule Backup Using Crontab

With the script verified, the next step is to automate it using crontab — Linux's built-in task scheduler. The cron job will execute the backup script every five minutes, simulating a frequent backup cycle for demonstration purposes.

5.1 — Opening the Crontab Editor

Command — open crontab editor

```
[user@parrot]~[/etc]
└─$ crontab -e
no crontab for user - using an empty one
Select an editor. To change later, run select-editor again.
 1. /bin/nano          <---- easiest
 2. /usr/bin/zile
 3. /usr/bin/nvim
 4. /usr/bin/mg
 5. /usr/bin/vim.tiny
 6. /usr/bin/codium

Choose 1-6 [1]: 3
crontab: installing new crontab
```

5.2 — Setting the Schedule

The following entry instructs cron to run the backup script every five minutes:

*Crontab entry — */5 **** schedule*

```
[user@parrot]~[/]
└─$ crontab -l
*/5 * * * * /home/user/rsync_scr_cl
```

5.3 — Updating Source Files

To demonstrate that the backup captures changes, new data is appended to the source files:

Command — adding new entries to source files

```
[user@parrot]~[/etc]
→ $echo "George" >> /home/user/S-PII.info/clients.txt
[user@parrot]~[/etc]
→ $echo "+1-555-000" >> /home/user/S-PII.info/phone_numbers.txt
```

5.4 — Verifying Backup Results

After waiting for the cron job to execute, the backup directories are inspected to confirm the snapshot was created and the changes were captured:

Output — (current_backup)¹ and (backup_history)² showing (old)¹ (new)² (files)¹ (timestamped directory)² with (updated info)¹ (previous file state)².

```
→ $ls -la /backups/current_backup
total 12
drwxrwxr-x 1 user user 74 May 13 17:52 .
drwxr-xr-x 1 user user 56 May 13 17:46 ..
-rw-rw-r-- 1 user user 32 May 13 17:50 cl_id.txt
-rw-rw-r-- 1 user user 31 May 13 18:23 clients.txt
-rw-rw-r-- 1 user user 59 May 13 18:23 phone_numbers.txt
[user@parrot]~[/etc]
→ $cat /backups/current_backup/clients.txt
John
Jane
Robert
Ivan
George
[user@parrot]~[/etc]
→ $ls -la /backups/backup_history
total 0
drwxr-xr-x 1 user user 114 May 13 18:28 .
drwxr-xr-x 1 user user 56 May 13 17:46 ..
drwxr-xr-x 1 root root 0 May 13 18:12 2026-05-13_18-12-14
drwxr-xr-x 1 root root 0 May 13 18:15 2026-05-13_18-15-38
drwxr-xr-x 1 root root 74 May 13 18:28 2026-05-13_18-28-10
```

Output — Timestamped directory in /backup_history, which contains updated files with previous info

```
[x]-[user@parrot]~[/etc]
→ $cat /backups/backup_history/2026-05-13_18-28-10/clients.txt
John
Jane
Robert
Ivan
```

Step 6

Disaster Recovery

Step 6 — Disaster Recovery

A backup system is only valuable if it can successfully restore data. This section simulates a complete data loss event and demonstrates full recovery from the backup.

6.1 — Simulating Data Loss

All files in the source directory are deleted using `rm -rf` to simulate accidental deletion, ransomware, or hardware failure:

Command — rm -rf simulating complete data loss

```
[user@parrot]~$ rm -rf /home/user/S-PII.info/*
[user@parrot]~$ ls /home/user/S-PII.info
[user@parrot]~$
```

Oops!

6.2 — Restoring from Backup

Data is restored from `/backups/current_backup` using `rsync`, which copies all backed-up files back to the original source location:

Command — rsync restore from current_backup to source directory

```
[x]-[user@parrot]~$ rsync -avz /backups/current_backup /home/user/S-PII.info
sending incremental file list
current_backup/
current_backup/cl_id.txt
current_backup/clients.txt
current_backup/phone_numbers.txt

sent 378 bytes  received 77 bytes  910.00 bytes/sec
total size is 122  speedup is 0.27
```

6.3 — Verifying the Restore

The restored directory is inspected to confirm all files are present and intact:

Output — source directory restored with all original files confirmed

```
[user@parrot]~$ ls /home/user/S-PII.info
current_backup
[user@parrot]~$ ls /home/user/S-PII.info/current_backup
cl_id.txt  clients.txt  phone_numbers.txt
```

In a production environment, we should always verify restored file contents and checksums — not just filenames — to confirm data integrity after recovery.

Step 7

Are There Other Options? rsync vs duplicity

Step 7 — Are There Other Options? rsync vs duplicity

rsync is an excellent tool for fast, transparent local backups. However, it has limitations — most notably the absence of built-in encryption. In any environment where data confidentiality is required (such as PII, financial records, or healthcare data), encryption at the backup level is not optional.

Two widely used alternatives worth knowing are **Déjà Dup** and **duplicity**. Both are suited for encrypted backups, with duplicity being the lower-level tool that Déjà Dup is actually built upon. Understanding duplicity is therefore more foundational.

Feature	rsync	duplicity
Primary purpose	File synchronisation	Encrypted backup archiving
Encryption	None built-in	Built-in GPG encryption
Backup format	Plain files — directly browsable	Compressed encrypted archives
Incremental backups	Manual implementation required	Native — fully automatic
Restore method	Simple rsync copy-back command	<code>duplicity restore command</code>
Storage efficiency	Stores complete copies each time	Stores only changes after first backup
Backup verification	Manual file inspection	Built-in collection-status command
Best used for	Fast local sync in trusted environments	Encrypted archiving for sensitive data

Key takeaway: **rsync** is the right tool when speed and simplicity matter in a trusted local environment. **duplicity** is the right tool when data confidentiality, integrity verification, and regulatory compliance are required — which in any scenario involving PII data, they always are.

The End.